

```

/**
 * =====
 * Slops Saloon Fantasy Football MVP (SSFFMVP)
 * Tuesday Morning Cron - v3 Production
 * =====
 * Schedule: 0 6 * * 2    (6:00 AM EST every Tuesday)
 * Deploy:   Railway Cron Service / Render Cron Job
 *
 * Three v3 hardening features fully implemented:
 *
 * [1] HUMAN-IN-THE-LOOP GATE
 *     Query filters WHERE followed = true so only moves the user
 *     actually executed in their league app feed the calibration model.
 *     followed = false/null moves → marked "not_executed", excluded.
 *
 * [2] REDIS CACHE LAYER (Upstash serverless)
 *     NFL scores cached 3600s per week - 100 users in the same
 *     league share one Sportradar API call, not 100.
 *     League outcomes cached 300s per league_id.
 *
 * [3] SUPABASE VAULT TOKEN DECRYPTION
 *     access_token + espn_s2 stored encrypted via pg_sodium Vault.
 *     Decrypted on read via vault_decrypt_secret RPC.
 *     Plaintext NEVER sits in any DB column.
 * =====
 */

"use strict";

const { createClient } = require("@supabase/supabase-js");
const Anthropic         = require("@anthropic/sdk");
const { Redis }         = require("@upstash/redis");

// [B][C] Import VORP + Scarcity from the Master Grade agent module
const {
  buildVORPTable,
  buildScarcityReport,
  fetchWithLocalFallback,
} = require("../ssffmvp_agents");

// — ENV VALIDATION —
const REQUIRED = ["SUPABASE_URL", "SUPABASE_SERVICE_KEY", "SPORTRADAR_API_KEY", "RED
for (const k of REQUIRED) {
  if (!process.env[k]) { console.error(`FATAL: missing env ${k}`); process.exit(1

```

```

}

const SPORTRADAR_BASE = "https://api.sportradar.com/nfl/official/trial/v7/en";
const CURRENT_SEASON = new Date().getFullYear();
const MODEL           = "claude-3-5-sonnet-latest";

// — CLIENTS —
const supabase = createClient(process.env.SUPABASE_URL, process.env.SUPABASE_SER
const anthropic = new Anthropic({ apiKey: process.env.ANTHROPIC_API_KEY });
const redis     = new Redis({ url: process.env.REDIS_URL, token: process.env.REDI

// — LOGGER —
const ts = () => new Date().toISOString().replace("T", " ").slice(0,19);
const log = {
  info: (...a) => console.log( `[${ts()}] `, ...a),
  ok:   (...a) => console.log( `[${ts()}] ✓`, ...a),
  warn: (...a) => console.warn( `[${ts()}] ⚠`, ...a),
  error: (...a) => console.error( `[${ts()}] ✖`, ...a),
  cache: (...a) => console.log( `[${ts()}] `, ...a),
  math: (...a) => console.log( `[${ts()}] `, ...a),
};

/* =====
[2] REDIS CACHE HELPERS
Key schema (all prefixed ssff: for easy Upstash console filtering):
  ssff:scores:{season}:{week}      TTL 3600s — stable post-Monday-night
  ssff:outcomes:{leagueId}:{week}  TTL 300s  — live until final whistle
  ssff:players:nfl                 TTL 86400s — roster changes once/day
===== */

const CACHE_KEYS = {
  scores: (s, w) => `ssff:scores:${s}:${w}`,
  outcomes: (lid,w) => `ssff:outcomes:${lid}:${w}`,
};

async function cacheGet(key) {
  try {
    const v = await redis.get(key);
    if (v !== null) { log.cache(`HIT ${key}`); return v; }
    log.cache(`MISS ${key}`); return null;
  } catch(e) { log.warn(`Redis GET error (${key}): ${e.message}`); return null; }
}

async function cacheSet(key, value, ttl) {
  try {
    await redis.set(key, typeof value === "string" ? value : JSON.stringify(value)
    log.cache(`SET ${key} TTL=${ttl}s`);
  } catch(e) { log.warn(`Redis SET error (${key}): ${e.message}`); }
}

```

```
}
```

```
/*
```

```
[3] SUPABASE VAULT - DECRYPT TOKEN
```

Tokens are stored via `vault.create_secret()` and only their `secret_id` UUID lives in `platform_connections.token_secret_id`. This RPC returns the plaintext only at query time, never stored.

One-time Supabase SQL setup:

```
create extension if not exists supabase_vault;
create or replace function vault_decrypt_secret(secret_id uuid)
returns jsonb language sql security definer as $$
    select jsonb_build_object('decrypted_secret', vault.decrypted_secret(secret_id))
    as decrypted_secret;

```

```
*/
```

```
async function decryptToken(secretId) {
  if (!secretId) return null;
  const { data, error } = await supabase.rpc("vault_decrypt_secret", { secret_id: secretId });
  if (error) throw new Error(`Vault decrypt failed for ${secretId}: ${error.message}`);
  return data?.decrypted_secret ?? null;
}
```

```
/*
```

```
[1] FETCH PENDING MOVES - HUMAN-IN-THE-LOOP GATE
```

The `.eq("followed", true)` line IS the gate. Without it, a user who tapped Accept but forgot to actually make the waiver claim in Sleeper would poison our calibration data.

`followed = null` → user never answered the FeedbackCard (rare edge case)
`followed = false` → user said "No - I skipped it" in FeedbackCard
`followed = true` → user confirmed they made the move → score this

```
*/
```

```
async function fetchPendingMoves() {
  const lastSunday = getMostRecentSunday();
  log.info(`[HITL] Fetching followed=true pending moves before ${lastSunday.toISOString()}`);

  // SCORE: only moves user confirmed they executed
  const { data: toScore, error: e1 } = await supabase
    .from("moves")
    .select(`id, user_id, week_num, season, move_type, headline,
      confidence, scoring, target_player, platform, league_id,
      outcome, followed, created_at,
      users ( id, email, push_token )`)
    .eq("outcome", "pending")
    .eq("followed", true) // ← THE HITL GATE
    .lt("created_at", lastSunday.toISOString())
    .order("created_at", { ascending: false });
}
```

```

if (e1) throw new Error(`Supabase HITL fetch failed: ${e1.message}`);
log.ok(`[HITL] ${toScore.length} executed move(s) queued for scoring`);

// ARCHIVE: moves user said they didn't follow (exclude from model)
const { data: notFollowed } = await supabase
  .from("moves")
  .select("id")
  .eq("outcome", "pending")
  .in("followed", [false])
  .lt("created_at", lastSunday.toISOString());

if (notFollowed?.length) {
  await supabase.from("moves")
    .update({ outcome: "not_executed", scored_at: new Date().toISOString() })
    .in("id", notFollowed.map(m => m.id));
  log.info(`[HITL] Archived ${notFollowed.length} not-executed move(s) - excluded`);
}

return toScore;
}

/* =====
[2] FETCH NFL SCORES - REDIS CACHED
1st call: Sportradar live → parse → cache 3600s
Calls 2-N: Redis HIT <5ms, zero API cost
===== */
async function fetchNFLScores(weekNum) {
  const key = CACHE_KEYS.scores(CURRENT_SEASON, weekNum);
  const hit = await cacheGet(key);
  if (hit) return typeof hit === "string" ? JSON.parse(hit) : hit;

  log.info(`[Sportradar] Live fetch Week ${weekNum} Season ${CURRENT_SEASON}`);
  const url = `${SPORTRADAR_BASE}/seasons/${CURRENT_SEASON}/REG/${weekNum}/schedule`;
  const res = await fetch(url);
  if (!res.ok) throw new Error(`Sportradar ${res.status} ${res.statusText}`);
  const data = await res.json();

  const playerScores = {};
  for (const game of data.week?.games || []) {
    if (game.status !== "closed") continue;
    extractPlayerStats(game, playerScores);
  }

  log.ok(`[Sportradar] Parsed ${Object.keys(playerScores).length} players`);
  await cacheSet(key, playerScores, 3600);
  return playerScores;
}

```

```
}
```

```
function extractPlayerStats(game, map) {
  const process = (team) => {
    for (const p of team?.rushing?.players || []) {
      const k = norm(p.name);
      map[k] = { ...(map[k]||{}), name:p.name, rush:(p.yards||0)*0.1+(p.touchdown
    }
    for (const p of team?.receiving?.players || []) {
      const k = norm(p.name); const b = (p.yards||0)*0.1+(p.touchdowns||0)*6;
      map[k] = { ...(map[k]||{}), name:p.name,
        rec_ppr:(b+(p.receptions||0)), rec_half:(b+(p.receptions||0)*0.5), rec_st
    }
    for (const p of team?.passing?.players || []) {
      const k = norm(p.name);
      map[k] = { ...(map[k]||{}), name:p.name,
        pass:(p.yards||0)*0.04+(p.touchdowns||0)*4-(p.interceptions||0)*2 };
    }
  };
  process(game.summary?.home);
  process(game.summary?.away);
}
```

```
const norm = (n='') => n.toLowerCase().replace(/[^a-z\s]/g,"").replace(/\s+/g,"_")
```

```
/* =====
[2] LEAGUE OUTCOMES - REDIS CACHED
===== */
```

```
async function fetchLeagueOutcomes(leagueId, weekNum) {
  const key = CACHE_KEYS.outcomes(leagueId, weekNum);
  const hit = await cacheGet(key);
  if (hit) return typeof hit === "string" ? JSON.parse(hit) : hit;

  try {
    const res = await fetch(`https://api.sleeper.app/v1/league/${leagueId}/matchu
    if (!res.ok) throw new Error(`Sleeper ${res.status}`);
    const matchups = await res.json();
    const groups = {};
    for (const m of matchups) (groups[m.matchup_id] = groups[m.matchup_id]||[]).p
    const outcomes = {};
    for (const [,pair] of Object.entries(groups)) {
      if (pair.length !== 2) continue;
      const [a, b] = pair;
      outcomes[a.roster_id] = (a.points||0) > (b.points||0);
      outcomes[b.roster_id] = !outcomes[a.roster_id];
    }
    await cacheSet(key, outcomes, 300);
  }
```

```

    return outcomes;
  } catch(e) { log.warn(`Sleeper outcomes failed ${leagueId}: ${e.message}`); ret
}

/* =====
EFFECTIVENESS SCORING – v4 VORP-Enriched
[B] VORP is now used to set the projection baseline instead of
the hardcoded 12.5 fallback. If a player had a high VORP going
into the week, the bar is higher – and if they clears it,
the eff score is proportionally stronger.
[A] Uses fetchWithLocalFallback so waiver data survives cloud outages.
===== */
async function scoreMove(move, playerScores, leagueOutcomes) {
  const playerKey = findBestMatch(norm(move.target_player || move.headline), Obj
  const playerData = playerKey ? playerScores[playerKey] : null;
  let eff = 30, outcome = "loss", detail = "Player data unavailable";

  if (playerData) {
    const actual = calcPts(playerData, move.scoring);

    // [B] VORP-adjusted projection: use historical pts_last3 if stored on move,
    // otherwise fall back to position baseline from the VORP engine.
    // This replaces the hardcoded 12.5 with a math-backed number.
    let proj = 12.5;
    try {
      const { data: waivers } = await fetchWithLocalFallback(
        move.league_id, "waivers",
        async () => {
          const r = await fetch(`https://api.sleeper.app/v1/league/${move.league_
            if (!r.ok) throw new Error(`Sleeper ${r.status}`);
          return r.json();
        }
      );
      const vorpTable = buildVORPTable(waivers, move.scoring);
      const norm_name = norm(move.target_player || "");
      const playerVorp = vorpTable.find(p =>
        norm(p.name).includes(norm_name.split("_").pop())
      );
      if (playerVorp) {
        // Projection = baseline + VORP (i.e. what we expected them to score)
        const { REPLACEMENT_BASELINES } = require("./ssffmvp_agents");
        const baselines = (REPLACEMENT_BASELINES?.[move.scoring] || {});
        const baseline = baselines[playerVorp.pos] || 12.5;
        proj = parseFloat((baseline + Math.max(0, playerVorp.vorp)).toFixed(2));
        log.math(`VORP projection for ${move.target_player}: ${proj} (baseline:${
      }
    } catch(e) {

```

```

    log.warn(`VORP projection failed, using default 12.5: ${e.message}`);
  }

  const ratio = actual / proj;
  if (ratio >= 1.15) { eff += 25; outcome = "win"; }
  else if (ratio >= 1.0) { eff += 15; outcome = "win"; }
  else if (ratio >= 0.85) { eff += 5; }

  const userWon = leagueOutcomes[move.user_id];
  if (userWon === true) { eff += 25; outcome = "win"; }
  if (userWon === false) { eff -= 10; }

  // Confidence calibration
  const c = move.confidence || 50;
  if (outcome==="win" && c>=75) eff+=20;
  if (outcome==="win" && c< 50) eff+=10;
  if (outcome==="loss" && c>=75) eff-=15;
  if (outcome==="loss" && c< 50) eff-= 5;

  eff = Math.max(0, Math.min(100, Math.round(eff)));
  detail = `${playerData.name} ${actual.toFixed(1)}pts (VORP proj:${proj}) - ${
    log.ok(`Scored ${playerData.name} | ${actual.toFixed(1)}pts | eff:${eff} | ${
  } else {
    log.warn(`No match for: "${move.target_player || move.headline}"`);
  }
  return { outcome, eff, detail };
}

const calcPts = (d, sc) => {
  const r = d.rush||0;
  return sc==="PPR" ? r+(d.rec_ppr||0)+(d.pass||0) : sc==="Half PPR" ? r+(d.rec_h
};

const findBestMatch = (target, keys) => {
  if (keys.includes(target)) return target;
  const last = target.split("_").pop();
  return keys.find(k => k.includes(last)) || null;
};

/* =====
  SAVE + NOTIFY
===== */
async function saveScoredMove(id, outcome, eff, result) {
  const { error } = await supabase.from("moves")
    .update({ outcome, eff, result, scored_at: new Date().toISOString() }).eq("id
  if (error) throw new Error(`DB update failed ${id}: ${error.message}`);
  log.ok(`Saved move ${id}: ${outcome} (${eff}%)`);

```

```
}
```

```
async function notifyUser(user, move, outcome, eff, detail) {
  if (!user?.push_token || !process.env.PUSH_SERVICE_URL) return;
  const body = `${outcome==="win"?`🏆`:"📄"} Wk${move.week_num}: ${eff}% - ${detail}`;
  try {
    await fetch(process.env.PUSH_SERVICE_URL, {
      method:"POST",
      headers:{"Content-Type":"application/json","Authorization":`Bearer ${process.env.PUSH_TOKEN}`},
      body: JSON.stringify({ token:user.push_token, title:`📄 Slops Saloon Result` });
    });
  } catch(e) { log.warn(`Push failed ${user.id}: ${e.message}`); }
}
```

```
/* =====
  SELF-IMPROVING LOOP - clean data only (followed=true)
  ===== */
```

```
async function updateSelfImprovingContext() {
  log.info("Updating self-improving context (followed=true only)...");
  const { data: moves, error } = await supabase.from("moves")
    .select("move_type, outcome, eff, confidence, scoring")
    .not("eff", "is", null)
    .eq("followed", true)
    .neq("outcome", "not_executed")
    .order("scored_at", { ascending: false })
    .limit(30);

  if (error || !moves?.length) { log.warn("Insufficient clean data for context update"); return; }

  const winRate = moves.filter(m=>m.outcome==="win").length / moves.length;
  const avgEff = moves.reduce((a,m)=>a+m.eff,0) / moves.length;
  const byType = moves.reduce((acc,m)=>{ (acc[m.move_type]=acc[m.move_type]||[]).push(m); return acc; }, {});
  const typeStats = Object.entries(byType).map(([t,ms])=>({
    type:t, winRate:`${(ms.filter(m=>m.outcome==="win").length/ms.length*100).toFixed(1)}%`,
    avgEff:`${(ms.reduce((a,m)=>a+m.eff,0)/ms.length).toFixed(1)}%`, n:ms.length
  })).sort((a,b)=>b.avgEff-a.avgEff);

  const hc = moves.filter(m=>m.confidence>=75);
  const hcWR = hc.length ? `${(hc.filter(m=>m.outcome==="win").length/hc.length*100).toFixed(1)}%` : 0;
  const best = typeStats[0];
  const worst = typeStats[typeStats.length-1];

  const prompt = `SLOPS SALOON HISTORICAL PERFORMANCE (n=${moves.length} clean experiments)
  Win rate: ${(winRate*100).toFixed(1)}% | Avg effectiveness: ${avgEff.toFixed(1)}%
  Best move type: ${best?.type} (${best?.avgEff}% eff, ${best?.winRate} wins, n=${best?.n})
  Weakest type: ${worst?.type} (${worst?.avgEff}% eff) - lower confidence here
  High-confidence (≥75) win rate: ${hcWR}%`;
```


All types: \${typeStats.map(t=>`\${t.type}:\${t.avgEff}%`).join(" | ")}

CALIBRATION: Adjust your confidence scores to match this history. Lean toward pro

```
const { error: uErr } = await supabase.from("system_context")
  .upsert({ key:"manager_agent_calibration", value:{ prompt, stats:{winRate,avg
if (uErr) log.warn(`Context upsert failed: ${uErr.message}`);
else log.ok(`Self-improving context saved - ${winRate*100}.toFixed(0)}% wins |
}
```

```
/* =====
HELPERS + MAIN
===== */
```

```
const getMostRecentSunday = () => {
  const d = new Date(); d.setUTCHours(0,0,0,0);
  d.setDate(d.getDate() - ((d.getDay()+7)%7||7)); return d;
};
const getCurrentNFLWeek = () => {
  const s = new Date(`${CURRENT_SEASON}-09-05`);
  return Math.max(1, Math.min(18, Math.floor((Date.now()-s.getTime())/(7*24*60*60
});
const groupBy = (arr, key) => arr.reduce((acc,i)=>{ (acc[i[key]]=acc[i[key]]||[])
```

```
async function main() {
  const t0 = Date.now();
  log.info("=====");
  log.info("Slops Saloon Cron v3 - start");
  log.info("=====");
  let scored=0, failed=0;
  try {
    const pending = await fetchPendingMoves();
    if (!pending.length) { log.info("No executed pending moves."); return; }
    const weekNum      = getCurrentNFLWeek() - 1;
    const playerScores = await fetchNFLScores(weekNum);
    for (const [leagueId, moves] of Object.entries(groupBy(pending, "league_id"))
      const leagueOutcomes = await fetchLeagueOutcomes(leagueId, weekNum);
      for (const move of moves) {
        try {
          const { outcome, eff, detail } = await scoreMove(move, playerScores, le
          await saveScoredMove(move.id, outcome, eff, detail);
          await notifyUser(move.users, move, outcome, eff, detail);
          scored++;
        } catch(e) { log.error(`Move ${move.id}: ${e.message}`); failed++; }
      }
    }
    await updateSelfImprovingContext();
  } catch(e) { log.error(`Fatal: ${e.message}\n${e.stack}`); process.exit(1); }
  log.ok(`Done in ${((Date.now()-t0)/1000).toFixed(1)}s - scored:${scored} failed
```

```

}

main().then(()=>process.exit(0)).catch(e=>{ console.error(e); process.exit(1); })

/*
DEPLOY (Railway):
  1. New project → Cron Service → connect GitHub
  2. Schedule: 0 6 * * 2
  3. Add env vars + Upstash Redis plugin (auto-sets REDIS_URL + REDIS_TOKEN)

PACKAGES:  npm i @supabase/supabase-js @anthropic-ai/sdk @upstash/redis

SUPABASE SCHEMA ADDITIONS:
  create extension if not exists supabase_vault;
  alter table moves add column if not exists followed boolean default null;
  alter table moves add column if not exists scored_at timestampz;
  -- outcome: 'pending' | 'win' | 'loss' | 'not_executed'

  create or replace function vault_decrypt_secret(secret_id uuid)
  returns jsonb language sql security definer as $$
    select jsonb_build_object('decrypted_secret', vault.decrypted_secret(secret_i
  $$;
*/

```